LAB # 15
Demonstrate the concept of Exception Handling.

Objective

To demonstrate the concept of Exception Handling.

Lab Goals

- Learn the concept of handling exceptions in Java programming.
- Gain proficiency in using try-catch blocks to manage exceptions effectively.
- Understand the role of finally blocks in exception handling and resource management.
- Learn to throw exceptions explicitly and handle checked exceptions using the throws clause.

Exception Handling

Exception handling in Java is a mechanism to handle runtime errors (exceptions) that occur during the execution of a program. Exceptions disrupt the normal flow of the program and can be caused by various factors such as incorrect input, hardware failures, or network issues.

Try-Catch Block

A try-catch block is used to handle exceptions in Java. Code that might throw an exception is enclosed within a try block, and the catch block catches and handles the exception if it occurs.

Syntax:

```
try {
    // code that might throw an exception e.g., division by zero, array index out of bounds, etc.
}
catch (ExceptionType e) {
    // code to handle the exception
    // e.g., logging, recovery, or displaying an error message
}
```

Example Code:

```
public class TryCatchExample {
    public static void main(String[] args) {
        try {
            int[] arr = new int[5];
            arr[7] = 10; // trying to access an index out of array bounds
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Exception caught: " + e);
        }
        System.out.println("Program continues after the try-catch block.");
    }
}
```

Output:

```
Exception caught: java.lang.ArrayIndexOutOfBoundsException: Index 7 out of bounds for length 5
Program continues after the try-catch block.
```

Try-Catch-Finally Block:

A try-catch-finally block is used to handle exceptions and also execute cleanup code (in the finally block) regardless of whether an exception is thrown or not.

Syntax:

```
try {
    // code that might throw an exception
}catch (ExceptionType e) {
    // code to handle the exception
}finally {
    // cleanup code that executes always // e.g., closing resources like files, database connections
}
```

Example Code:

```
import java.io.*;
public class TryCatchFinallyExample {
    public static void main(String[] args) {
        BufferedReader br = null;
        try {
            br = new BufferedReader(new FileReader("input.txt"));
            String line;
            while ((line = br.readLine()) != null) {
                System.out.println(line);
            }
        } catch (FileNotFoundException e) {
            System.out.println("File not found: " + e);
        } catch (IOException e) {
            System.out.println("IOException: " + e);
        } finally {
            try {
                if (br != null)
                    br.close();
            } catch (IOException e) {
                System.out.println("Error in closing the BufferedReader: " + e);
            }
            System.out.println("Finally block executed.");
        }
    }
}
```

Output:

```
File not found: java.io.FileNotFoundException: input.txt (No such file or directory)
Finally block executed.
```

Throwing an Exception using “throw” keyword:

In Java, throw keyword is used to explicitly throw an exception from a method or block of code.

Syntax:

```
throw throwableInstance;
```

Example Code:

```
public class ThrowExample {
    public static void main(String[] args) {
        try {
            validateAge(15);
        } catch (Exception e) {
            System.out.println("Exception caught: " + e);
        }
    }

    public static void validateAge(int age) {
        if (age < 18) {
            throw new IllegalArgumentException("Age must be at least 18.");
        }
        System.out.println("Valid age.");
    }
}
```

Output:

```
Exception caught: java.lang.IllegalArgumentException: Age must be at least 18.
```

“throws” Clause:

The **throws** clause in Java specifies that a method can throw exceptions. It declares the exceptions that a method can throw but does not handle them itself.

Syntax:

```
void method() throws ExceptionType1, ExceptionType2, ... {
    // method code
}
```

Example Code:

```
public class ThrowExample {
    public static void main(String[] args) {
        try {
            validateAge(15);
        } catch (Exception e) {
            System.out.println("Exception caught: " + e);
        }
    }

    public static void validateAge(int age) {
        if (age < 18) {
            throw new IllegalArgumentException("Age must be at least 18.");
        }
        System.out.println("Valid age.");
    }
}
```

Output:

```
Exception caught: java.lang.IllegalArgumentException: Age must be at least 18.
```

Exercise

A. What will be the output?

```
public class Example1 {
    public static void main(String[] args) {
        try {
            int[] numbers = {1, 2, 3};
            System.out.println(numbers[4]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Array index out of bounds.");
        } finally {
            System.out.println("Finally block executed.");
        }
    }
}
```

B. Create a Java program to handle ArithmeticException when dividing by zero using try, catch, and finally blocks.
Expected Output:

```
Cannot divide by zero.
Finally block executed.
```

C. Write a program in which you have to take amount and if amount is less than 10000 so exception occurs with some greeting message i.e. “You are unable to get the requested withdraw amount” using throw Keyword else “Amount Successfully withdraws”.

D. Write a program in which you have to take amount and if amount is less than 10000 so exception occurs with some greeting message i.e. “You are unable to get the requested withdraw amount” using throws Keyword else “Amount Successfully withdraws”.